RSI Catch-Up: What Is Improving, How It Learns, and What to Study

A structured reading guide to research published or substantially documented between **3 August and 4 November 2026**, written for your background in nanoGPT, SFT, DPO, GRPO, DGM, AlphaEvolve, and execution-grounded automated research.

Background: the next questions after DGM and AlphaEvolve

You already know how an LLM can propose a code change, run it, and retain a better program. The useful next question is where the system gets a *better way to conduct that search*. Three bottlenecks organize the recent work:

1. **Learning from experience.** A research agent can collect successful trajectories without learning to reproduce their useful behavior. MASS trains coordination into the model; CARE teaches a deployed model from diagnosed failures.
2. **Producing useful experience.** A fixed task pool eventually limits learning. SPADE learns an environment generator alongside the solver. The generator must create tasks that are challenging and learnable.
3. **Conducting research efficiently.** AIDE² improves the research harness; Dream-RSI learns how to allocate experiments from recorded histories; HEMO evolves the procedure that invents algorithmic components. Exo lets useful harness changes take effect during ongoing work.

These extend the lineage you studied: **AlphaEvolve** searches for better programs; **DGM** makes the agent's own code a search target; the newer methods ask how training experience, research decisions, or the model performing those decisions can improve. The August Anthropic study adds another question: can automated research produce useful model-training methods that survive evaluation beyond the search objective? See your RSI lineage map and execution-grounded research guide for the earlier foundation.

Keep three objects separate throughout:

Object	In your nanoGPT setting	What improvement would mean
The research product	A training script or trained model	It trains faster or performs better.
The research procedure	How the agent proposes, selects, and runs experiments	It finds useful changes more reliably or cheaply.
The learning process	How the researcher uses experience to change itself	Its acquired behavior helps subsequent research.

An **improver** is the procedure that takes a current system and produces candidate successors. It can be code around a frozen model, a learned model, or both. Useful progress can occur while an outer optimizer remains fixed. Stronger claims—that successive researchers become increasingly effective improvers—need direct comparisons of their improvement ability. The evaluation companions explain those comparisons without making that strongest claim the only contribution worth studying.

Connections to your nanoGPT/SFT work below are proposed translations of the ideas. They are not experiments these papers performed on your setup.

One recommended order

This is a **catch-up priority for your RSI research**, judged by what each work adds beyond your existing knowledge, its importance to the research agenda, and the evidence supporting its mechanism. It is not an objective ranking of field-wide impact or a ranking of benchmark scores. Availability on one H800 affects the suggested exercise, rather than silently determining the scientific priority.

Priority	Main reading	The new idea to understand
1	MASS	Successful collaboration becomes training data for the model that collaborates again.
2	SPADE	The learner also learns to create useful training environments.
3	AIDE²	Optimize the researcher's search and context-management code.
4	Dream-RSI	Reuse expensive experiment histories to test research strategies cheaply.
5	HEMO	Evolve the procedure that discovers useful algorithmic operators.
6	Exo	Activate durable harness improvements while the agent keeps working.
7	August Anthropic automated alignment researchers	Test whether automated research discovers transferable training methods.
8	CARE	Turn diagnosed deployment failures into model updates.

The order changes substantially from the earlier guide. MASS and SPADE introduce learning mechanisms beyond the self-editing loop you already know. AIDE² remains the most directly relevant research-harness case. Dream-RSI and HEMO precede Exo because they address the research procedure more directly; Exo contributes runtime architecture and amortized efficiency. The last two studies demonstrate valuable model-improvement applications with research/update machinery that remains externally designed.

Evaluation papers have a different job from these methods. Read **Audit's claim/evidence distinction** alongside AIDE², **SelfImprove-Bench** before designing your own evaluation, and **False Frontiers** when building a synthetic-data loop. S3Gym and CogRSI answer more specific questions about experience storage and the validity of learned rules. They receive separate explanations below. Model releases and specialized applications follow as optional branches.

1. MASS: learn from a better-organized team

UC Berkeley and Sakana AI; October 2026. Recursive Self-Improvement through Multi-Agent Self-Supervision.

The problem

In your SFT work, someone supplies the demonstrations. A self-improving model needs a way to create demonstrations that teach it something useful. Asking it to repeat and judge its own answers can preserve its existing mistakes. MASS explores a different source of supervision: several copies of the model can divide a difficult task, exchange intermediate findings, and produce a better trajectory than an unstructured attempt.

The contribution and mechanism

MASS combines **workflow search** with **training on successful executions**. A workflow specifies roles, call order, and what information each worker must return.

1. The current model executes candidate workflows, compares their code, reports, and other output files, and proposes revised workflows.
2. Selected workflows produce several task executions. The system keeps the better trajectories, including the coordinator's decisions and bounded worker conversations.
3. SFT trains the model on those trajectories while omitting the optimized workflow from its input. The model must learn the behavior rather than merely follow a supplied plan.
4. The updated model becomes executor, evaluator, and workflow optimizer for the next cycle. The paper runs two complete cycles.

The conceptual advance is carrying an organizational improvement into the model that performs the next round. A useful pattern can become default behavior: the model chooses to delegate without receiving the original workflow instructions.

What the evidence means

On **ScienceAgentBench**, scientific coding tasks checked by executable tests, success rises from **28.8% to 31.7%** after two cycles. A more targeted test freezes the executor: workflows generated by the updated model beat the original model's workflows in **64%** of comparisons. That helps separate better workflow generation from stronger execution.

A generic instruction to coordinate already elicits much of the base model's ability. This narrows the result toward internalizing useful coordination, rather than establishing a wholly new ability. Results on other coding benchmarks also do not improve uniformly. The next research question is which part of the learned behavior transfers and whether the evaluator remains reliable as the model changes.

Why it is first, and what it means for your research

This is the closest bridge between your DGM knowledge and your hands-on SFT experience: improve the way a model works, convert that experience into weights, then reuse the model in the next improvement round. It comes before SPADE because it directly connects learned behavior to the research roles you want to improve.

For your own agent, you could collect successful experiment-selection and repair trajectories, train on them with the original strategy instructions removed, and test whether the model now performs those behaviors unaided. Include a simple “coordinate carefully” prompting control so you can distinguish learned default behavior from capability already available through prompting.

Read: §3, pp. 4–5; §4.2, pp. 6–7; Appendix C.3–C.4, pp. 18–20. Code is available, with documented differences from the reported runs. The published first training cycle used six H100s; a small H800 experiment would test the idea at a different scale.

2. SPADE: learn what to practice next

Liu, Yu, et al.; August 2026, revised August. SPADE.

The problem

Your GRPO experiments start with a task distribution and a reward. As the model learns, that distribution may become too easy, remain too hard, or lack the situations needed for the next skill. Scaling self-improvement therefore requires a way to produce new, useful experience. Simply asking a generator for harder questions can produce impossible or repetitive tasks.

The contribution and mechanism

SPADE trains one shared model in two roles: **environment designer** and **solver**. The designer creates an executable setting with persistent state, available actions, transitions, and rewards. Think of a small board game: its environment specifies the board, legal moves, how a move changes the board, and how to score success.

1. The designer reads a source passage and memory of previous designs, then writes a Python environment and a private hint.
2. The solver tries the environment with and without the hint.
3. The difference between those outcomes estimates a learning opportunity: difficult without help, approachable with help. The designer's reward also targets an appropriate difficulty band.
4. GRPO updates the shared weights. Both the next solver and the next designer therefore change through learning.

Source material supplies breadth; the reward and memory help select what is worth practicing. This moves the creation of training experience into the learned system, while the training algorithm and reward construction remain human-designed.

What the evidence means

For the 30B model, the unweighted mean of eight benchmark percentage scores rises from **50.2 to 58.3 on a 0–100 scale**. The scores measure math/science answer accuracy, code pass rate, and procedural-game win rate. The strongest fixed-environment training comparison reaches **53.0**. The important comparison is the latter: adapting the training environments adds value beyond continuing training on a fixed collection.

Four columns are procedural-reasoning categories, so the average should not be read as an equal gain across general intelligence. Component studies support the importance of source grounding, memory, and joint adaptation. Checkpoint selection and incomplete accounting for environment generation/hint trials still limit a claim about total compute efficiency.

Why it is second, and what it means for your research

SPADE addresses a central bottleneck that DGM-style code search does not itself solve: where the next useful learning experiences come from. That earns it a place above another harness-optimization study. This does not mean it demonstrates stronger recursive acceleration than AIDE[2]; it advances a different part of the learning system.

For your research, a useful first step is to inspect whether generated environments require the skill their reward claims to measure. Later, you could test a small designer/solver loop where the designer is rewarded for creating learnable failures, rather than merely difficult ones. This connects directly to your GRPO knowledge without requiring you to relearn policy optimization.

Read: §§4–7, especially Algorithm 1, p. 7, and Table 1, p. 9; inspect one generated environment in the appendix. Code and checkpoints exist. The published launchers target an eight-GPU node.

3. AIDE²: improve the researcher, not only its current experiment

Weco AI; September technical report following a July announcement. Recursive Self-Improvement of AI Research Agents.

The problem

An autoresearch agent can improve `train.py` while remaining a poor experimenter. It may keep refining one promising idea, fail to start genuinely different directions, or bury useful evidence in an ever-growing conversation. AIDE² makes the code governing that research behavior the target of optimization.

The contribution and mechanism

There are two nested searches. The inner agent searches for better research outputs; the outer agent searches for a better inner agent.

1. The outer agent edits the inner research harness.
2. Each candidate harness receives AI R&D tasks and a fixed budget. It proposes changes, executes experiments, and selects an output.
3. Private evaluation grades those outputs. Useful harness revisions become starting points for subsequent edits.
4. The final harness is tested on additional research tasks. The research model's weights remain fixed.

The discovered changes explain the contribution more clearly than its aggregate grade. A greedy search becomes a mix of drafting strategies, with trial allocation favoring promising strategies while still trying uncertain ones. The agent starts fresh branches when progress stalls and gives each role a bounded context containing relevant failures.

What the evidence means

In an external weather-forecasting task, the agent starts from a physics-based forecasting program and searches for numerical changes that make multi-day forecasts more accurate under a fixed research budget. Both evolved harnesses find a strong family of changes in **all three runs**; the initial and human-designed harnesses reach a comparable solution in **at most one of three**. The task family was absent from harness selection. This supports more reliable discovery on an unfamiliar problem.

The paper also makes the evolved researcher the *outer* improver and compares it with the human-designed researcher from the same starting harness. Three runs per arm give no clear advantage. That second experiment asks a stronger question than whether the final research harness is useful; its inconclusive result should be kept separate from the positive transfer result.

Why it is third, and what it means for your research

AIDE² is the clearest direct example for your research-agent agenda. It follows MASS and SPADE because you already know self-editing search from DGM; those two teach additional routes for learning from, and generating, experience. AIDE² then shows which research behaviors are actually worth changing.

You could compare old and revised researchers on the same untouched nanoGPT starting script, hardware budget, and fresh tasks. Examine whether broader search and better failure memory produce useful experiments more reliably. A stronger inherited training script would otherwise hide whether the *experimenter* improved.

Read: §2, pp. 3–5; §3.3, p. 7; §3.5–3.6, pp. 8–11; the weather task in Appendix A, pp. 24–25. An exact public implementation was not established from the paper and announcement.

4. Dream-RSI: use old experiments to improve the next search

Google and collaborators; September paper, October revision/code release. Dream-RSI.

The problem

Suppose you want to compare two research strategies: keep refining the current best experiment, or explore another direction. Testing each strategy normally requires paying for another research run. Improving the research process can therefore cost as much as the research itself.

The contribution and mechanism

Dream-RSI turns completed research histories into replay environments for testing **allocation policies**—rules deciding what to try, how much to parallelize, and when to stop.

1. A fixed coding agent conducts a real search and records a tree of attempts, outputs, scores, and costs.
2. The tree becomes a replay environment. A candidate policy sees only the history revealed so far and chooses which recorded branch to reveal next.
3. Another coding process revises policy code and tests alternatives cheaply on those trees.
4. The selected policy controls the next real search, whose history enlarges the replay pool.

This is replay of observed outcomes. It cannot predict what an experiment absent from the tree would have done. Its advantage is making many alternative research decisions testable without rerunning the expensive experiments already recorded.

What the evidence means

One task asks the agent to make a **Lasso numerical solver** faster while preserving correctness. With the same Gemini-Flash backbone, Dream-RSI uses **1,879 discovery-agent calls**, compared with **3,200** for a fixed policy. The resulting solver averages **2,350 ms rather than 2,517 ms** on six held-out datasets, improving five of six.

These are two distinct benefits: fewer agent decisions during research, and somewhat faster code produced by that research. Agent calls are not total dollar cost, and the experiment does not establish the variation across many independent complete policy-learning runs.

Why it is fourth, and what it means for your research

The reusable insight is how to make learning a research strategy cheaper. That maps directly to your expensive GPU experiments. HEMO also changes search control, but evaluates revised control through actual downstream searches; Dream-RSI first extracts more value from work already paid for. This is a priority judgment about the mechanism, not a claim that Dream-RSI wins a common benchmark against HEMO.

You could preserve complete nanoGPT experiment histories, including failures and resource use, and compare policies that reveal those outcomes in different orders. Reserve new online runs for testing whether the selected policy still helps outside the recorded tree.

Read: §§2–3, pp. 3–6; Figure 4, p. 8; Appendix D, pp. 22–27. Code includes exploration, replay, and policy rewriting; not every experiment in the paper is included.

5. HEMO: improve the procedure that invents algorithms

Singapore Management University, NTU, and collaborators; October 2026. HEMO.

The problem

In evolutionary algorithm design, an LLM can invent an **operator**: a small component that changes candidate solutions, such as a rule for modifying routes. But the process that invents these operators often stays manual. The best research decisions early in a search may become poor decisions once useful operators and failure evidence accumulate.

The contribution and mechanism

Keep three levels distinct: a route is a solution; a route-changing rule is an operator; the procedure that invents such rules is the design harness. HEMO evolves that third level.

1. A scheduler selects a harness module, such as history retrieval or parent-operator selection.
2. A proposer specifies a revision; another agent implements it in Python.
3. The revised harness conducts operator search. The system measures the quality of the resulting solutions and the search runtime.
4. It activates useful revisions and retains experience for further changes. The outer scheduler, evaluator, and model weights remain fixed.

The generated rules are concrete: retrieve different experience when the search stagnates, combine complementary parents, or switch between exploration and refinement.

What the evidence means

The tasks involve several objectives, so a good result is a set of useful trade-offs rather than one universally best solution. For transfer, the selected harness is frozen and tested on a related problem with **empty memory**—for example, moving from a two-objective route problem to a three-objective version.

Across nine transfer settings, HEMO is **best in five and second in four** on the paper's measure of trade-off quality. Empty target memory makes this evidence about a reusable search procedure. However, the equal final candidate budgets exclude the preceding harness search, which can take tens to hundreds of hours.

Why it is fifth, and what it means for your research

HEMO is a direct study of improving how discovery proceeds. Its inspectable modules and related-task transfer make it more central to that question than Exo's runtime efficiency results. It follows Dream-RSI because the domains are specialized and the cost of evaluating each harness remains substantial.

For your own experiments, separate history retrieval, parent selection, and change generation into explicit functions. You could then test whether a learned retrieval rule helps on a new training task with fresh memory. That separates a reusable research habit from simply carrying over a notebook of successful recipes.

Read: §III, pp. 4–6, especially Figure 2; Tables I–II, p. 8; generated modules in Appendix F; budgets in Appendix H, pp. 36–38. A standalone public reproduction repository was not established.

6. Exo: let useful improvements survive inside a running agent

Krentsel et al.; UC Berkeley and collaborators; October 2026. Exo.

The problem

A long-running agent repeatedly pays a model to rediscover API details, assemble context, or execute the same diagnostic sequence. A reusable tool could remove that waste. But replacing the code controlling a running agent raises a systems question: how does the successor continue with the same history and task environment?

The contribution and mechanism

Exo separates editable policy code from protected runtime mechanisms.

1. The policy controls tools, model calls, context construction, and memory retrieval.
2. During work, the agent edits that policy and requests activation of a replacement.
3. Protected mechanisms preserve an append-only history and versioned artifacts. A separate guardian checks the replacement before activation.
4. The new policy resumes work with the retained history and environment. Model weights stay fixed.

For example, a repeated sequence of model-mediated API calls can become one direct reusable tool. The contribution is allowing that change to pay off during ongoing work, rather than waiting for a separate offline optimization stage. The default guardian checks whether the replacement builds; additional behavioral guarantees require additional checks.

What the evidence means

AppWorld simulates requests involving ordinary apps. After learning on fifty scenario families, Exo is tested on new variants of those same families. Across three seeds, both Exo and the identical policy without editing achieve **96.9% success**. Total model cost **including learning and reflection falls from $57.36 to $32.70**, a 43% reduction.

The important result is amortization: the learned tools save enough repeated work to repay their creation cost. A one-off quest can fail to repay that overhead. Transfer here concerns scenario variants, so it does not establish the same saving on unrelated tasks.

Why it is sixth, and what it means for your research

Exo supplies useful architecture for a persistent research agent. It follows Dream-RSI and HEMO because its clearest result concerns runtime adaptation and efficiency, whereas those methods directly optimize research allocation or discovery control.

For your agent, the transferable design is to preserve experimental evidence while allowing the code that summarizes logs, diagnoses failures, or builds context to improve. Count the cost of acquiring each improvement and the number of later runs that reuse it. An impressive local optimization may have little value if it is never needed again.

Read: §§3–4, pp. 3–5; §5.4 and Table 1, pp. 8–9. Code is public; release documentation still describes unfinished features.

7. Automated alignment researchers: discover training methods that transfer

Anthropic and collaborators; August 2026, September revision. Automated Researchers Can Mitigate Well-characterized Alignment Failures.

The problem

An automated researcher can produce plausible post-training proposals without discovering anything that improves a real model. This study chooses measurable failure families—such as deception, sycophancy, and susceptibility to injected instructions—so proposed training methods can be executed and tested beyond the feedback used to develop them.

The contribution and mechanism

The system searches over model-training methods, including data and objectives, rather than only generating answers.

1. A shared literature review gives the researchers relevant prior work.
2. For each failure family, five fixed Claude researchers propose methods, write implementations, and share findings through a forum and leaderboard.
3. Each candidate trains a small target model on one GPU for about thirty minutes. A monitor checks prohibited data use; evaluators return safety and capability measurements.
4. Promising methods receive further search and separate transfer evaluations.

The research agents remain fixed. The outputs being improved are training recipes and target-model weights. This makes it a study of automated AI research, with particularly concrete evaluation of its products.

One sycophancy case study makes the output tangible. The discovered recipe creates a teacher by shifting the target model's internal activations to reduce its tendency to echo the user's opinion, then uses KL-based self-distillation to teach that behavior to the unsteered target model. This recipe analysis uses the sycophancy search suite; its ablations are not additional held-out tests.

What the evidence means

For **each of ten separately studied failure families**, its best method improves a separate previously unseen benchmark. These are family-specific methods, rather than one universal safety recipe. Later behavioral and larger-model tests select among leading methods using the unseen benchmark; for those choices it becomes validation, while the behavioral audit supplies an additional test.

When post-training an early production checkpoint, the automated method closes **65% of the measured gap from that baseline to ideal safety**; the lab's released production model closes **72%**. This shows that automated post-training research can produce useful methods

in a demanding setting. It concerns the measured failure families. Instruction-following also declines in the main experiments; a noisy capability gate can allow substantial regressions. The experiment therefore teaches a safety/capability trade-off rather than unchanged general competence.

Why it is seventh, and what it means for your research

This is important evidence that AI-for-AI research can produce transferable training methods. It comes after the earlier mechanisms because it does not train or evolve the researcher itself. Its application evidence still earns a place in the main route.

It connects closely to your SFT/DPO/GRPO experience: you could compare bounded searches over data alone with searches allowed to change the training objective. Keep a final evaluation that did not select the method, and measure the skills a targeted improvement may damage. The paper's iterative automated search and human one-shot ideas have different opportunities, so their comparison does not isolate matched-resource research ability.

Read: §§2–3, pp. 3–5; §5, pp. 7–11; the capability gate in Appendix A.4, p. 23; data-versus-objective ablation in Appendix D.4, pp. 37–40. Code is available, with training infrastructure left to the user. This is the recent follow-up to April's weak-to-strong demonstration.

8. CARE: learn from a failure after finding its actual cause

Keeta/Meituan and university collaborators; November 2026. CARE.

The problem

A production agent's bad conversation is not automatically a good training example. The fault may be a broken tool or outdated business rule. Even when the model made the mistake, copying a few corrected answers can destabilize it. CARE addresses the path from an observed failure to an appropriate model update.

The contribution and mechanism

CARE combines failure attribution with **on-policy self-distillation**. Its teacher gains an advantage from diagnosis and hindsight, while starting from the same weights as the student.

1. An inspection process identifies the first erroneous decision and attributes it to the model, a tool, a skill, or the harness.
2. Component faults go to repair. Model faults receive a reviewed diagnosis and repair.
3. A frozen copy of the current model becomes a teacher that sees this privileged information. The student sees only the original decision-time context and generates a response.
4. The teacher scores that sampled response token by token. A KL-based update teaches the student the better-informed behavior. Tests and human review govern promotion.

The learning signal comes from what the model can do *when given a sound explanation of its failure*. Normal inference must reproduce the fix without receiving that explanation.

What the evidence means

Across five updates, the offline inspected Bad Case rate falls from **6.42% to 1.97%**. A separate executable simulator checks whether the requested business operation was actually performed; accuracy rises from **93.0% to 94.1%**. Saying that a refund succeeded is insufficient if no refund was executed.

The distinction matters in a small-update control: DPO produces few inspected errors by handing conversations to humans, while task completion collapses. That diagnoses a failure of the local objective, not a general verdict against DPO. Business-policy changes and component repairs also accompany the weight updates, so the full production improvement cannot be attributed solely to the distillation loss.

Why it is eighth, and what it means for your research

CARE adds a production-tested learning signal and a disciplined way to route failures. It is lower than the research-procedure papers because the inspection, repair, and update procedure remains externally designed and the domain is customer service.

For your post-training experiments, you could give a frozen teacher a verified failure diagnosis, withhold it from the student, and test whether the student learns a repair that survives normal inference. First establish that the failure belongs to the model. An out-of-memory error or broken evaluator should not become evidence for changing the model's reasoning.

Read: §5, pp. 13–15; §6, pp. 15–19; Tables 9–10, p. 28. An exact production-code or weight release was not established.

Evaluation companions: how to design the experiment

These papers answer different questions. They should influence your experimental design rather than be treated as five interchangeable warnings.

Audit: what claim would your experiment actually test?

Problem and contribution. A better training script, a repeatably useful research loop, and a better researcher are different achievements. Auditing the Product, Not the Recursion maps each claim to an experiment and examines how often papers report it. Its most reusable contribution is that map.

How to use it. Evaluate the final product on untouched tasks to test product quality. Audit intermediate versions to test accumulation. Rerun the whole search to test repeatability. To compare improvers, apply the old and new researcher to the same starting artifact under the same budget, then evaluate their outputs. These tests answer different questions; one rising curve cannot replace all of them.

What the evidence adds. Only four of the 26 sampled studies claiming a better improver report matched-budget comparisons from identical starting artifacts. That counts experimental designs, not four established breakthroughs. The census uses AI-assisted coding outside its human-reviewed subsets, so its precise prevalence estimates deserve caution.

Why learn it now. Read its claim table before interpreting AIDE²'s two different experiments. For your nanoGPT agent, it explains why you must hold the starting script fixed when comparing experimenters. Read §§3–4, pp. 4–8, then Table 4, p. 11. It is an experimental-design foundation, rather than another algorithm competing for a main-route rank.

SelfImprove-Bench: how far does a learned change travel?

Problem and contribution. New seeds, harder tasks, and a different capability are different forms of transfer. SelfImprove-Bench makes those distinctions explicit while keeping the backbone fixed and allowing prompts, memory, or code to change.

How it works. Agents refine themselves on training games. Saved versions are then tested on exact replay, unseen instances, higher difficulty, another game in the same capability family, and another capability family. These are called G0 through G4, respectively. An edit trace records which permitted components actually changed.

What the evidence means. Sixteen of eighteen method/game combinations improve on unseen instances, while eight regress on exact replay. Transfer and forgetting can coexist. None of the 915 retained edits in the main run changes the improver, even though the architecture permits such changes. The result is about actual behavior, rather than the system's advertised editable surface.

Why learn it now. This is the most directly reusable evaluation blueprint for your next experiment. Translate its distinctions into new seeds, datasets, and research tasks; its game families are not themselves a measure of scientific competence. Small evaluation groups limit fine method rankings. Read pp. 4–11 and the configuration provenance, which explains differences from historical runs.

False Frontiers: a separate evaluator can inherit the same mistake

Problem and contribution. A proposer invents a question and answer; a solver learns that answer; the solver then judges later proposals. Their agreement can rise because both inherited the same false answer. False Frontiers contributes a concrete intervention, CrossFit, rather than simply recommending a second judge.

How it works. Split source documents into two groups. An auxiliary solver trained on questions from group A evaluates proposals derived from B, and another does the reverse. The main solver still learns from all admitted examples. Splitting individual questions would leave shared document-level errors on both sides.

What the evidence means. For the 4B system, the mean reference-answer match score across seven search/QA evaluations rises from **40.0% to 48.8%** relative to the coupled loop. This metric, Cover-EM, checks whether a normalized reference answer appears in the extracted answer. Matching on the same wrong answer falls from **6.1% to 3.0%**. Full runs reserve larger GPU-hour budgets; actual utilization was not measured. The result supports breaking an error-feedback path, while leaving equal-cost superiority unresolved.

Why learn it now. Read this after MASS or SPADE if you plan to create synthetic training data. Track the ancestry of the data used to teach a judge, not merely its model name. Read §§2–5, pp. 3–9, and the cost analysis, pp. 16–17. No code release was established from the paper.

S3Gym: where should experience be stored?

Problem and contribution. Raw history, compressed lessons, and weight updates preserve different information. S3Gym compares those choices in seven games where agents explore and judge their own experience, while independent evaluation retains executable ground truth.

How it works. The same experience can be appended as trajectories, distilled into a strategy memory, or used for SFT. Evaluation uses separate seeds and stricter settings. This separates whether the model approves of its experience from whether reusing that experience helps.

What the evidence means. Different storage choices win in different games. A specific rule for comparing current and next Tetris pieces can help, while generic advice to avoid obstacles omits the geometry required for Snake. In one Qwen3-8B training experiment, Plants-vs-Zombies survival falls from **23 turns to 6** after updates. The retained model can become worse at a skill after training on additional self-selected experience; this result alone does not isolate the cause.

Why learn it now. Use this when deciding whether your experiment logs should remain raw evidence, become explicit research heuristics, or become SFT examples. Keep the final retained policy and its regressions visible rather than reporting only its best moment. Limited episodes make detailed rankings fragile. Read §§4–6, especially pp. 15 and 18–19; project.

CogRSI: useful advice can still express a false rule

Problem and contribution. An agent can improve task accuracy while learning a rule that is logically invalid. CogRSI evolves short theory-of-mind principles and then audits their meaning using executable counterexamples.

How it works. A reader tracks who observed story events and uses principles to resolve uncertain answers. A builder revises those principles from failures. After evolution, independent translators turn a rule into code and test it in a formal story world. For example, equating presence with knowledge fails when the present character is asleep.

What the evidence means. Of 330 rules with a decisive semantic audit, 286 are invalid and 44 valid. Additional rules outside that subset are ambiguous or inapplicable. Some invalid rules still improve answer accuracy because the model may compensate for or ignore them. The audit tests a compiled interpretation of the written rule, rather than directly revealing the model's internal reasoning.

Why learn it now. This is a narrower companion than Audit or SelfImprove-Bench. Use it when turning research memories into reusable guidance: “larger batches improve training” needs conditions and counterexamples. Read §§4–5, pp. 3–9, and audit details, p. 19. The advertised repository was inaccessible as of this guide's update.

Optional branches: choose by the question you want to pursue

ScienceCUA: turn manuals into skills that survive a fresh session

Problem and idea. A manual contains facts and syntax, but an agent needs a procedure it can execute. ScienceCUA separates procedural Skills from shared command References. It constructs them from documentation, diagnoses failures during execution, and accepts repairs only when they work in a fresh environment without the exploratory conversation.

Evidence and significance. Across six scientific applications, Grok 4.6 averages **37.47%** task success without the library, **44.70%** with constructed skills, and **54.59%** after repair. The trajectory explains two sources of value: extracting operational knowledge and correcting it through use. Each evolution task starts from the same initial library; accepted changes are merged afterward. The study therefore evaluates a batch of transferable repairs, rather than successive globally improved generations. A stronger backbone gains much less.

Why this is a branch. This may be the best practical next read if you want to build research-agent skills. It sits outside the main route because the learned object is a domain-specific library, while the model and improvement procedure remain fixed. Try separating an experiment framework's workflow from its command reference, then test the resulting skill without chat history. Read §§3–5 and Appendix I. The release currently covers construction; evolution and benchmark code are planned.

RSI-Env: make the successor conduct the next round

Problem and idea. Improving a model with a fixed external teacher does not test whether the model can continue the process. RSI-Env supplies persistent files, training/inference services, version lineage, and successor registration. In its Self setting, the accepted model–harness pair takes over the next round. Registration checks whether the successor runs.

Evidence and significance. In one knowledge-directed run, factual-question accuracy rises **82→85→88%**: first after a harness edit, then after the successor creates 48 training examples and updates weights. That demonstrates continuity across different update types. In the same run, graduate-level science-question accuracy falls **65.2→53.0%**. A functioning handoff can therefore carry both gains and regressions forward.

Why this is a branch. Read it when implementing successor takeover and model/harness lineage. Its contribution is infrastructure for asking stronger learning questions; the short trajectories and unequal utilized resources do not establish a generally effective improvement algorithm. You could implement the lineage/state transitions around one GPU without reproducing its 64-A800 system. Read §3, pp. 4–6, Table 2, p. 7, and §4.5, p. 9. A public reproduction release was not established.

Meta^n: let the next layer inspect both the failures and their cause

Problem and idea. Repeated reflection can keep fixing individual errors without noticing a shared cause. Meta ^ n gives a fixed updater both the execution traces and the context/helper code that produced them. It adds another layer of guidance and callable helpers; later layers can revise the earlier guidance.

Evidence and significance. Its assignment-problem example repeatedly fails by importing unavailable SciPy routines. A layer notices the shared dependency problem and supplies usable guidance/helpers. Another example keeps a useful label-cleaning function while rolling back an over-prescriptive instruction. The experiments support useful transfer of context and helpers, while the best result across an archive of specialized chains is different from the result of one deployable chain. More layers can also interfere and regress.

Why this is a branch. The implementable lesson is to expose the *code behind previous advice* as well as logs. This is relevant to your research memory design, but the updater and weights remain fixed, and the depth ablation does not fully match total compute. It comes after the methods that directly learn research control. Read §§2–3, pp. 3–12, and Appendix C; code.

RSI-Anything: improve the predictor that decides what to test

Problem and idea. You can afford to run only some proposed GPU experiments. A predictor could prioritize them, but its predictions may become unreliable as proposals change. RSI-Anything combines an action proposer with an outcome predictor, using observed outcomes to update components and checking whether the resulting pair still works together.

Evidence and significance. The clearest coding comparison holds the 7B coder fixed. Hidden-test success is **26.3%** with a static verifier, **29.2%** with predictive selection, and **31.2%** after predictor adaptation. The gain comes from better choices about which code patches to test. Sustained joint improvement of proposer and predictor remains a broader research target. Its released chemistry demonstration replays measured outcomes from a fixed pool.

Why this is a branch. Read it if you want a learned surrogate for expensive experiment outcomes. It is below Dream-RSI and HEMO because the clean evidence concerns particular subloops. For your setup, test whether a predictor ranks *new* experiments usefully and compare its overhead with the GPU time saved. Read §§2.3–2.5 and the coding experiment, pp. 11–13; inspect the release protocol before interpreting its laboratory claims.

Safin-1.5: test which behaviors survive further training

Problem and idea. A safety intervention can disappear during subsequent ordinary training. Safin-1.5 combines an architectural intervention with a safety-repair loop. REX moves targeted knowledge into removable experts; later training uses the full structure, after which deployment removes those experts. ReSI discovers attacks, tries repair recipes, and promotes models through safety/utility gates anchored to the initial baseline.

Evidence and significance. After identical further training, hazardous-domain answer accuracy rebounds to **81.7%** for direct unlearning but remains **40.8%** after REX's removal. Here lower accuracy on the targeted hazardous-domain test measures retained removal. General-task accuracy has a trade-off. The paper reports one ReSI round and leaves longer retention trajectories open. Its separate optional proof harness checks conclusions under formal premises; it does not certify that those premises correctly capture natural language or that future training updates are safe.

Why this is a branch. This is a concrete model release worth pairing with your MoE and post-training knowledge. Its relevance is retained-behavior testing and architecture, rather than established long-run recursive safety. Read REX and its matched-training comparison, pp. 7–8 and 19–21; ReSI, pp. 9–12; proof scope, pp. 13–16.

Recursive Flint: inspect an actual AI-assisted model-development product

Problem and idea. Efficient coding models require compatible choices across architecture, weight representation, and inference kernels. Recursive's October **Flint** release is a product of its AI-assisted development program, with weights and serving code. It is a model artifact, rather than a paper introducing a reproducible research-improvement algorithm.

Evidence and significance. The model card specifies **39.8B total parameters and about 2.5B active per token**. The technical release attributes architecture search, quantization recipes, and specialized kernels to Eureka. One idea is to quantize a relatively costly output head and recover quality through distillation, then optimize kernels for that representation. These contributions can reinforce one another across the stack.

Why this is a branch. Flint updates the older picture of an RSI company with no inspectable model output. Study it for model/inference co-design, which connects to your nanoGPT and MoE background. The public artifacts do not isolate Eureka's contribution from human work or expose the complete researcher. Active parameters describe per-token computation; all experts still need storage. Read the authored model card and the release's development section, then inspect rec. The documented local engine recipe targets supported Macs with at least 48 GB of memory.

SPAR: store a strategy together with the conditions that justify it

Problem and idea. A sales agent needs both a particular merchant's history and strategies reusable across merchants. SPAR maintains those separately. After an interaction, a fixed model assesses whether the outcome informs the strategy or reflects an external cause, then updates a structured playbook. The weights and editing procedure stay fixed.

Evidence and significance. A live merchant experiment reports **15.8% relative conversion uplift** over direct prompting. A frozen-playbook variant already gains **12.6%** over that baseline; the additional relative improvement is about **2.8%** over the frozen variant. This distinguishes the value of the overall system from the contribution of continued playbook evolution. The LLM's explanation of an outcome is still an attribution hypothesis.

Why this is a branch. Borrow its representation of conditional advice, rather than treating every successful run as a universal rule. In your experiments, a GPU crash should not count against a learning-rate strategy. The sales domain and fixed updater make this a memory-engineering application beyond the main RSI route. Read §3, pp. 2–4, and Tables 2–3, p. 6. A public deployment-code release was not established.

EpiAgent: make a belief useful by testing its predictions

Problem and idea. In bargaining or poker, an agent must infer another player's hidden preferences and act on that inference. A plausible belief paragraph is not enough. EpiAgent alternates training explicit belief updates and belief-conditioned actions in self-play. Beliefs earn reward by helping predict the next observation the agent receives, including the opponent's response; actions earn the game reward.

Evidence and significance. In a modified poker setting, replacing only the trained planner's belief input gives average utility **0.049 with random beliefs, 0.181 with learned beliefs, and 0.232 with the true persona**. These are game rewards, not win percentages. The intervention supports that useful belief information improves decisions, with room remaining above the learned beliefs.

Why this is a branch. It offers a learning signal and a clear intervention for your multi-agent RL interests. The fixed self-play algorithm trains interaction behavior rather than improving a research procedure. Read §4, pp. 5–7, and Table 3, p. 10. Its full training used eight A100s; a one-H800 exercise would need a smaller, explicitly scoped experiment. A complete public training release was not established.

Turn the reading into one research experiment

You do not need to reproduce a large paper to investigate its central idea. Choose one changed object and keep the others fixed.

1. **Research-control experiment:** inspired by Dream-RSI/HEMO, compare two experiment-allocation or history-retrieval policies around the same fixed coding model and initial nanoGPT script. Keep complete costs and failed runs. Use replay to screen ideas, then test on new online runs.
2. **Learning-signal experiment:** inspired by MASS/CARE, compare ordinary successful-trajectory SFT with trajectories generated using coordination or privileged diagnosis. Remove that extra scaffolding at evaluation. Include the simple prompting baseline and tests for skills that might regress.
3. **Experience-generation experiment:** inspired by SPADE, ask whether an adaptive generator supplies more learnable tasks than a fixed collection under a declared budget. Inspect what the executable reward actually checks and preserve a separate final test.

For any choice, specify the claim first: a better final artifact, a repeatable improvement loop, a transferable learned procedure, or a better improver. Save starting states and intermediate successors so the stronger comparison remains possible. With one H800, narrow tasks, code inspection, and controlled small experiments are credible starting points; the reported full training systems generally require more hardware and experience-generation compute.

The original surveys, AlphaEvolve, Self-Rewarding Language Models, and April's weak-to-strong study remain background for this update. TTCS began in January and has October revisions; it is outside this guide's selected new-mechanism route, rather than being treated as a newly introduced October method.